AI-Assisted Code Migration Platform

Core Plan of Action, Architecture & Roadmap

Focused on Verified Python / Node / C → Rust Migrations

1. Executive Overview

This document outlines a practical plan to build an AI-powered code migration platform. The core value is not simply rewriting code with AI, but producing verified, high-performance Rust equivalents with strong equivalence proofs, performance reports, and human review gates.

The platform takes existing source code (Python, TypeScript/Node, C, etc.) and outputs clean, idiomatic Rust while proving behavioral equivalence and measuring performance gains.

2. High-Level Architecture

The system is designed with clear separation of concerns so models, languages, or verification strategies can be swapped without major rewrites.

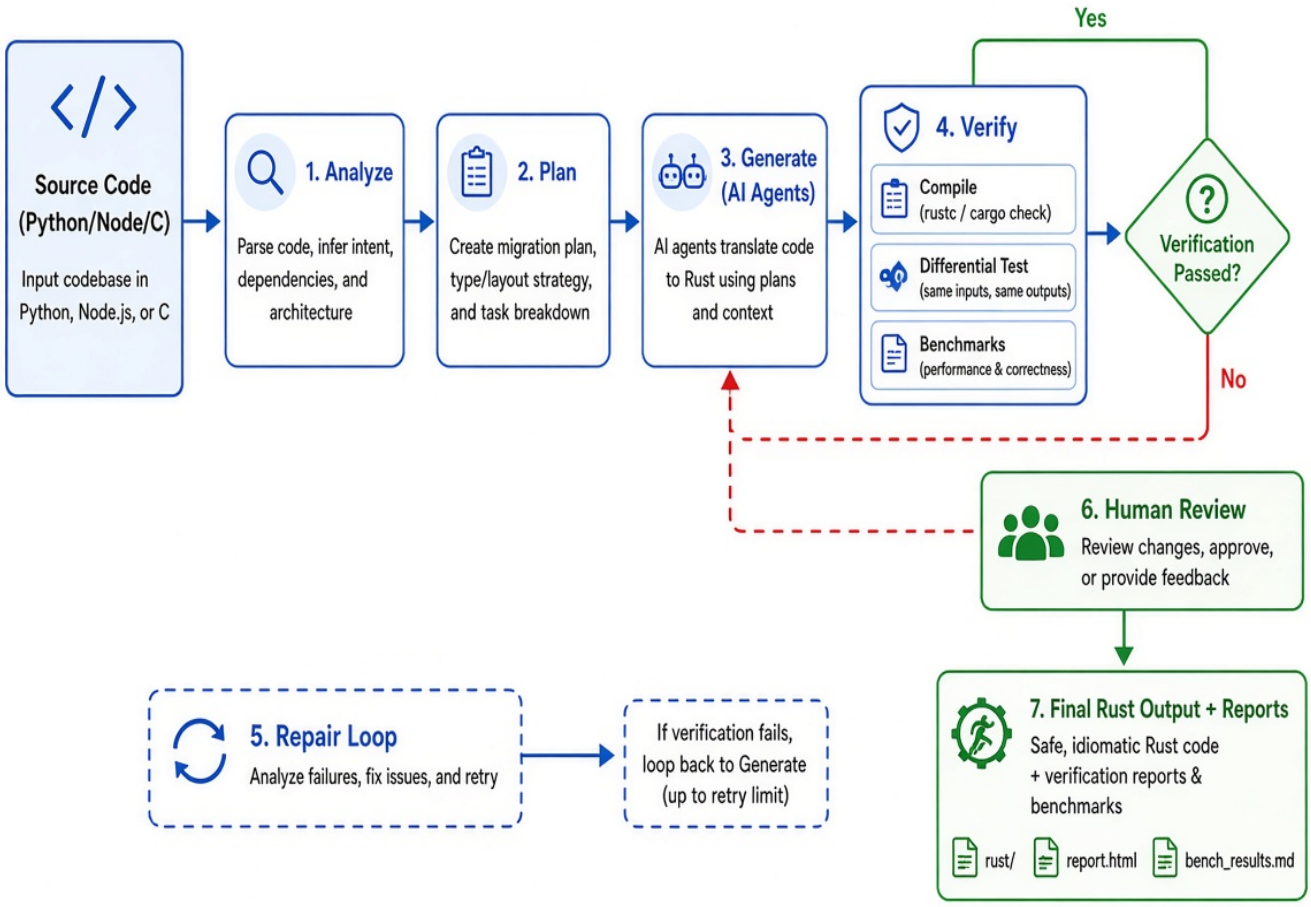


Figure 1: High-Level System Architecture

Main Layers:

- **Interface Layer** — CLI + Web dashboard for submitting repos, monitoring progress, and reviewing results.
- **Orchestration Layer (Python)** — Agent swarms, planning, repair loops, and state management.
- **Verification & Execution Layer (Rust)** — Sandboxes, differential testers, compilers, and benchmarks (the trust engine).
- **AI Layer** — Multi-provider model routing, prompt management, and tool calling.
- **Storage & Reporting** — Artifacts, equivalence reports, performance deltas, and audit logs.

3. Migration Pipeline (Core Flow)

Every migration runs through a controlled pipeline with automatic repair and human gates.

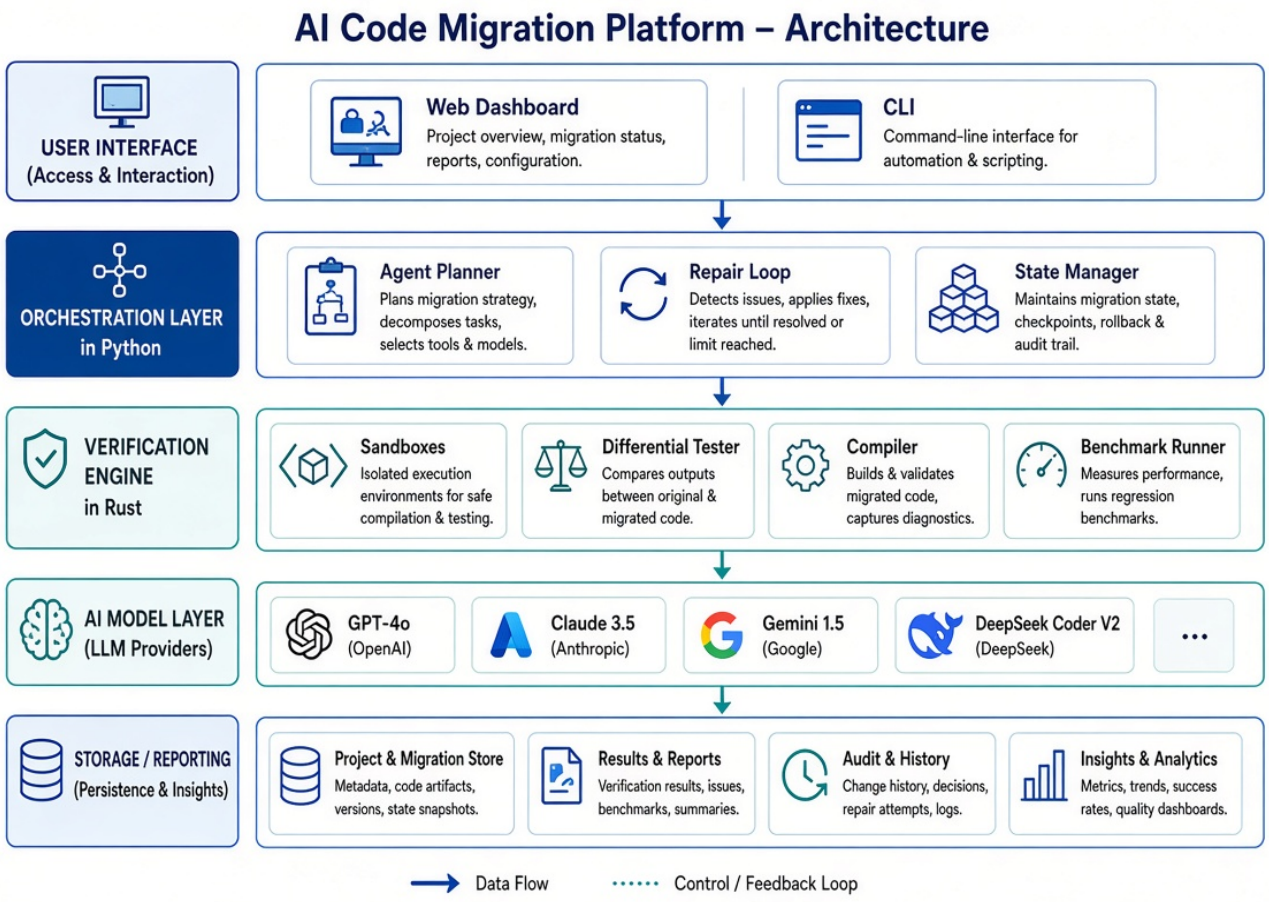


Figure 2: End-to-End Migration Pipeline

- **1. Ingest & Analyze** — Clone repository, build dependency graph, identify hot paths and module boundaries.
- **2. Plan** — AI generates a migration plan (priority order, independent modules, risk assessment).
- **3. Generate** — Agents produce Rust candidates for each module.
- **4. Verify** — Compile, differential testing (same inputs → same outputs), unit/integration tests, and performance benchmarks.
- **5. Repair** — On failure, feed diagnostics back to the agent for automatic repair (up to N attempts).
- **6. Human Gate** — Critical modules or low-confidence results require human review.
- **7. Output** — Clean Rust project + equivalence report + performance comparison + remaining risk list.

4. Recommended Technology Stack

Layer	Technology	Rationale
Orchestration & Agents	Python + LangGraph / custom agents	Best ecosystem for AI agents and rapid iteration
Verification Engine	Rust	Speed, memory safety, and low resource use for sandboxes and testing

Sandboxes	Firecracker microVMs or WASM	Strong isolation for untrusted generated code
Differential Testing	Custom Rust tool	Byte-level or semantic output comparison
Frontend	Next.js + CLI	Web for teams, CLI for developers
AI Models	Multi-provider gateway	Avoid lock-in; route by cost and quality
Storage	Postgres + S3-compatible	Metadata, artifacts, and reports

5. Milestone Division & Roadmap

Phase 0: Foundation (4–6 weeks)

Goal: Prove the core verification loop works on small examples.

- Working differential tester (Python vs Rust on simple functions).
- Basic agent that rewrites a function and gets repaired until tests pass.
- Success metric: 10/10 small modules migrate with zero behavioral difference.
- Deliverable: Side-by-side demo with proof of equivalence.

Phase 1: MVP CLI (8–12 weeks)

Goal: Single language pair (recommend Python → Rust) usable by technical users.

- CLI that accepts a folder or Git repository.
- Full pipeline for modules under ~500 lines.
- HTML/Markdown report with equivalence results and performance numbers.
- Open-source the verification core for credibility.
- Success: 5–10 external developers successfully migrate a real small service.

Phase 2: Web Platform Beta (12–16 weeks)

Goal: Team-friendly product.

- Web dashboard (Git connect, live progress, review interface).
- Multi-module support with dependency ordering.
- Human review gates and comment system.
- Basic authentication and project history.
- Success: First paid pilot customers.

Phase 3: Production Hardening & Expansion (ongoing)

Goal: Enterprise-ready product with expanded language support.

- Node/TypeScript → Rust support.
- Incremental migration for large codebases.

- Private deployment options (VPC / on-prem).
- Audit logs, SLAs, and team roles.
- Success: Repeatable revenue from mid-market or enterprise customers.

6. Immediate Next Actions (Week 1–2)

1. Pick the first language pair (Python → Rust recommended due to demand and AI support).
2. Build the smallest possible verification loop (one function in, one function out, prove they match).
3. Decide open-source vs proprietary strategy early (open-sourcing the verifier is usually smart).
4. Identify 5–10 potential design partners (startups or teams already facing Python/Node performance issues).

7. Key Risks to Watch Early

- Verification quality — this is the make-or-break feature.
- Agent reliability on real-world (not toy) code.
- Cost of AI tokens at scale.
- Trust — companies will only pay if the proofs are convincing.

Document generated as a practical planning guide. Architecture and pipeline diagrams included.